

CS204 – Algorithms & Data Structures Laboratory

Dr. V. Vijaya Saradhi,
Dept. Of CSE,
IIT Guwahati

saradhi@iitg.ac.in



Week #07: STL & Linked Lists



Overview



- **Standard Template Library**
- **Linked Lists**



Standard Template Library



Introduction



- The Standard Template Library (STL) is one of the most powerful features of C++.
- It provides a rich set of template-based **data structures and algorithms** that greatly simplify common programming tasks.
- The STL is part of the C++ standard library and is designed to be both efficient and flexible.

Introduction



- **Standard Template Library (STL)** has the following important elements
- **Containers:** These are data structures that store collections of objects.
- **Iterators:** These are objects that point to elements inside containers and help traverse through the elements.
- **Algorithms:** These are a set of functions that operate on containers using iterators. They include operations like sorting, searching, reversing, etc.
- **Functors (Function Objects):** These are objects that can be used as though they were functions or function pointers.

Containers



- Containers are objects that store collections of data. They provide various ways to organize and access data efficiently.
- Sequence containers
- Associative containers
- Unordered containers



Sequence Containers

- `std::array`: Fixed-size array
- `std::vector`: Dynamic array that can grow in size.
- `std::list`: Doubly linked list allowing fast insertions and deletions.
- `std::deque`: Double-ended queue, allowing fast insertion and deletion from both ends.
- `std::forward_list`: Singly linked list.

Associative Containers



- These containers store data in a way that allows quick retrieval using keys.
- `std::set`: Stores unique elements in a sorted order.
- `std::map`: Stores key-value pairs, with unique keys sorted in order.
- `std::multiset`: Stores multiple instances of elements, sorted.
- `std::multimap`: Stores key-value pairs with non-unique keys.

Unordered Containers



- These containers store data in a hashed manner to allow quick access.
- `std::unordered_set`: Stores unique elements but with no specific order.
- `std::unordered_map`: Stores key-value pairs, no specific order.
- `std::unordered_multiset`: Similar to `unordered_set` but allows duplicates.
- `std::unordered_multimap`: Similar to `unordered_map` but allows multiple keys.



Arrays

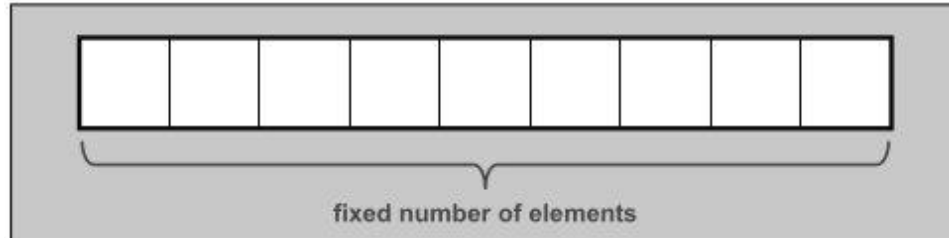


Arrays



- An array - an instance of the container class `array<>`
- models a static array.
- `#include<array>` header file
- An array is a sequence of elements with constant size.
- You can neither add nor remove elements to change the size.
- Only a replacement of element values is possible.

Arrays



Arrays



- Un-initialized array declaration

```
// Un-initialized array x containing 4 elements  
std::array<int,4> x;
```

Arrays



- initialized array declaration

```
// Initialize array y containing 4 elements with 0  
std::array<int,4> y = {};
```

Arrays



- initialized array declaration

```
// Initialize array c1 containing 5 elements with specified values  
std::array<int,5> c1 = { 42, 377, 611, 21, 44 };
```


Arrays



- Partial initialization of array

```
// Partial initialization of array.  
// First element 42; Remaining elements 0.  
std::array<int,10> c2 = { 42 };
```

Arrays - Operations



- Accessing using `[]` operator

```
cout << c1[0] << endl; // First element  
cout << c1[1] << endl; // Second element  
cout << c1[2] << endl; // Third element  
cout << c1[3] << endl; // Fourth element  
cout << c1[4] << endl; // Fifth element
```



Arrays - Operations

- Accessing using **at()** function
- Access elements safely (throws exception if out of bounds)

```
cout << c1.at(0) << endl; // First element  
cout << c1.at(1) << endl; // Second element  
cout << c1.at(2) << endl; // Third element  
cout << c1.at(3) << endl; // Fourth element  
cout << c1.at(4) << endl; // Fifth element
```

Arrays - Operations



- Accessing first and last elements

```
cout << c1.front() << endl; // First element  
cout << c1.back() << endl; // Last element
```

Arrays - Operations



- Copying array elements using copy constructor

```
array<int, 5> arr1 = {1, 2, 3, 4, 5};  
array<int, 5> arr2;  
  
arr2 = arr1;
```





Arrays - Operations

- Accessing array using iterators
- `arr.begin()`: Returns an iterator to the first element.
- `arr.end()`: Returns an iterator to the past-the-end element.

```
std::array<int, 5> arr = {1, 2, 3, 4, 5};

std::cout << "Array elements: ";
for (auto it = arr.begin(); it != arr.end(); it++)
{
    std::cout << *it << " ";
}
std::cout << std::endl;
```



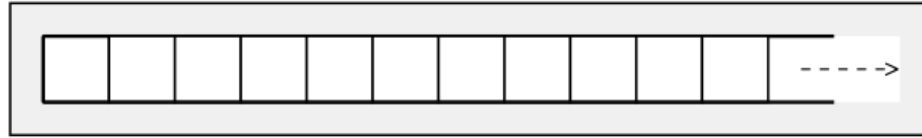
Vectors



Vectors



- A vector models a dynamic array.
- A vector is an abstraction that manages its elements with a dynamic C-style array



- `#include<vector>` header file

Vector



- Declare an empty vector

```
std::vector<int> vec1;
```

- Declare a vector of size 5 elements

```
std::vector<int> vec2(5);
```

- Declare a vector of size 5 with default values

```
std::vector<int> vec3(5, 100);
```

Vector



- Declare & initialize using array

```
std::vector<int> vec4 = {1, 2, 3, 4, 5};
```

- Initializing by copying

```
std::vector<int> vec5(vec4);
```

Vector - Operations



- Accessing using [] operator

```
cout << vec4[0] << endl; // First element  
cout << vec4[1] << endl; // Second element  
cout << vec4[2] << endl; // Third element  
cout << vec4[3] << endl; // Fourth element  
cout << vec4[4] << endl; // Fifth element
```

Vectors - Operations



- Accessing using **at()** function
- Access elements safely (throws exception if out of bounds)

```
cout << vec4.at(0) << endl; // First element  
cout << vec4.at(1) << endl; // Second element  
cout << vec4.at(2) << endl; // Third element  
cout << vec4.at(3) << endl; // Fourth element  
cout << vec4.at(4) << endl; // Fifth element
```

Vectors - Operations



- Accessing first and last elements

```
cout << vec4.front() << endl; // First element  
cout << vec4.back() << endl; // Last element
```

Vectors - Operations



- Adding five elements to vector of size 5 resulting in vector of size 10

```
cout << vec4.push_back(10) << endl; // 6th element  
cout << vec4.push_back(20) << endl; // 7th element  
cout << vec4.push_back(30) << endl; // 8th element  
cout << vec4.push_back(40) << endl; // 9th element  
cout << vec4.push_back(50) << endl; // 10th element
```

Vectors - Operations



- Removing five elements from vector of size 10 resulting in vector of size 5

```
cout << vec4.pop_back() << endl; // 10th element  
cout << vec4.pop_back() << endl; // 9th element  
cout << vec4.pop_back() << endl; // 8th element  
cout << vec4.pop_back() << endl; // 7th element  
cout << vec4.pop_back() << endl; // 6th element
```

Vectors - Operations



- Copying vector elements using copy constructor

```
std::vector<int> vec1 = {1, 2, 3, 4, 5};  
  
std::vector<int> vec2;  
  
vec2 = vec1;
```




Vectors - Operations

- Accessing vector using iterators
- Iterator **i** is vector iterator
- **vec4.begin()**: Returns an iterator to the first element.
- **vec4.end()**: Returns an iterator to the past-the-end element.

```
vector<int> vec4 = {1, 2, 3, 4, 5};  
vector<int>::iterator i;  
  
for( it = vec4.begin(); i != vec4.end(); i++)  
{  
    cout << *i << endl;  
}  
cout << endl;
```

Vectors - Operations



- **vec4.size()**: Obtains vector size

```
vector<int> vec4 = {1, 2, 3, 4, 5};  
  
cout << vec4.size() << endl;
```

- **vec4.capacity()**: Returns the current allocated storage of the vector (which can be larger than the size).

```
vector<int> vec4 = {1, 2, 3, 4, 5};  
  
cout << vec4.capacity() << endl;
```



Vectors - Operations

- **vec4.resize(10):** Resizes the vector to specified size

```
vector<int> vec4 = {1, 2, 3, 4, 5};  
  
cout << vec4.resize(10) << endl;
```

- **vec4.clear():** Removes all elements from the vector.

```
vector<int> vec4 = {1, 2, 3, 4, 5};  
  
cout << vec4.clear() << endl;
```

Vectors - Operations



- **vec4.empty():** Checks if the vector is empty

```
vector<int> vec4 = {1, 2, 3, 4, 5};  
  
cout << vec4.empty() << endl;
```

- **vec4.swap(vec5):** Swaps vector 4 with vector 5

```
vector<int> vec4 = {1, 2, 3, 4, 5};  
vector<int> vec5 = {6, 7, 8, 9, 10};  
  
vec4.swap (vec5) ;
```



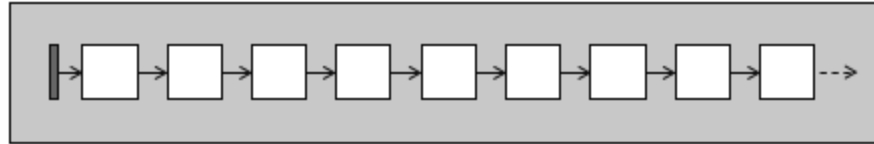
forward_list



forward_list



- A forward list manages its elements as a singly linked list



- To use a forward list, you must include the header file `<forward_list>`
- The elements of a forward list may have any type `T`.
- provides forward-only traversal
- We will learn

forward_list



- **Declaring `std::forward_list`**
- **Accessing Elements**
- **Adding Elements**
- **Removing Elements**
- **Using Iterators with `std::forward_list`**
- **Other Useful Member Functions**



Declaring forward_list

- **Declare empty list of integers**

```
forward_list<int> flist_1;
```

- **Initialize five number of elements with default value 0**

```
forward_list<int> flist_2(5);
```

- **Initialize five number of elements with default value 100**

```
forward_list<int> flist_3(5, 100);
```

- **Initializing using array/list**

```
forward_list<int> flist_4 = {1, 2, 3, 4, 5};;
```


Declaring forward_list



- Copy Initialization

```
forward_list<int> flist_5(flist_4);
```



Adding elements to forward_list





Adding elements to forward_list

- `push_front()`: Adds an element to the beginning of the list.

```
forward_list<int> flist = {10, 20, 30, 40};  
  
flist.push_front(5);  
// flist: 5, 10, 20, 30, 40
```



Adding elements to forward_list

- **push_front():** Adds an element to the beginning of the list.

```
forward_list<int> flist = {10, 20, 30, 40};  
  
flist.push_front(5);  
// flist: 5, 10, 20, 30, 40
```

- **insert_after():** Inserts elements after a specified position.

```
forward_list<int> flist = {10, 20, 30, 40};  
  
auto it = flist.begin();  
// Iterator pointing to 10  
++it; // Iterator now points to 20  
  
flist.insert_after(it, 7);  
// List contains {10 20 7 30 40}
```



Removing elements from forward_list





Removing elements from forward_list

- Remove first element

```
forward_list<int> flist = {5, 8, 7, 10, 20, 30, 40};  
  
// Remove first element  
flist.pop_front();
```



Removing elements from forward_list

- Remove specified element

```
forward_list<int> flist = {5, 8, 7, 10, 20, 30, 40};  
  
// it points to 5  
auto it = flist.begin();  
  
// Iterator points to 8  
++it  
  
flist.erase_after(it);  
  
//8 is removed from flist
```



Removing elements from forward_list

- Remove specified value

```
forward_list<int> flist = {5, 8, 7, 10, 20, 30, 40};  
  
flist.remove(20);  
  
//20 (all instances of 20) is removed from flist
```




Removing elements from forward_list

- **Conditional remove**

```
forward_list<int> flist = {5, 8, 7, 10, 20, 30, 40};  
  
flist.remove_if([](int x) { return x % 2 == 0; });  
  
// removes all even elements  
// Lambda function is written inside remove_if function
```



Removing elements from forward_list





Types of iterators

- **iterator:** Allows modification of elements.
- **const_iterator:** Allows read-only access to elements.
- **before_begin():** Returns an iterator to the position before the first element (useful for insertions).
- **end():** Returns an iterator to the position past the last element.

Iterator



- Iterator

```
forward_list<int> flist = {5, 8, 10, 30, 40};

forward_list<int>::iterator it = flist.begin();

for ( ; it != flist.end(); it++)
{
    cout << *it << endl;
}
cout << endl;
```

Const Iterator



- `const_iterator`

```
forward_list<int> flist = {5, 8, 10, 30, 40};

forward_list<int>::const_iterator cit = flist.begin();

for ( ; cit != flist.end(); cit++)
{
    cout << *cit << endl;
}
cout << endl;
```

Iterator



- **before_begin**: Returns an iterator to the position before the first element (useful for insertions).

```
forward_list<int> flist = {5, 8, 10, 30, 40};

forward_list<int>::const_iterator cit = flist.begin();

auto before = flist.before_begin();
flist.insert_after(before, 3);

//flist = 3, 5, 8, 10, 30, 40
```



Common Member Functions in forward_list





Common Member Functions

- **empty()** Checks if the list is empty.

```
forward_list<int> flist = {5, 8, 10, 30, 40};  
  
cout << flist.empty();
```




Common Member Functions

- **clear()** Removes all the elements from the list

```
forward_list<int> flist = {5, 8, 10, 30, 40};  
  
cout << flist.clear();
```



Common Member Functions

- **distance()** If you need the size, you can use `std::distance` with iterators.

```
forward_list<int> flist = {5, 8, 10, 30, 40};  
  
int c = distance(flist.begin(), flist.end());  
cout << c;
```



Common Member Functions

- `unique()` Removes consecutive duplicate elements.

```
forward_list<int> flist = {1, 2, 2, 3, 3, 3, 4};  
  
flist.unique();  
//flist becomes: 1, 2, 3, 4
```



Common Member Functions

- **sort()** Sorts the elements in ascending order.

```
forward_list<int> flist = {4, 3, 2, 1};  
  
flist.sort();
```



Common Member Functions

- **reverse()** Reverses the order of elements.

```
forward_list<int> flist = {4, 3, 2, 1};  
  
flist.reverse();  
//flist becomes: 1, 2, 3, 4
```



Common Member Functions

- `merge()` Merges two sorted `forward_list` into one

```
forward_list<int> flist_1 = {1, 3, 5, 7};  
forward_list<int> flist_2 = {2, 4, 6, 8};  
  
flist_1.merge(flist_2);  
//flist_1 becomes: 1, 2, 3, 4, 5, 6, 7, 8
```



Singly Linked List Implementation



Singly Linked List



- Node declaration

```
class Node
{
    public:
        int data;
        Node* next;
        Node(int value)
        {
            data = value;
            next = nullptr;
        }
};
```


Singly Linked List Class



- Node declaration

```
class LinkedList
{
    private:
        Node* head;
    public:
        LinkedList()
        {
            head = nullptr;
        };
        void insert(int val);
        void delete_node(int val);
        void print();
        ~LinkedList();
};
```



Questions?





Thank You

